

Illustration: building a bookstore API

Note that your project for the lessons comes after this illustration.

You are a backend developer tasked with building an API for a small bookstore. The store needs an API to manage its inventory of books. The system should allow users to:

1. View all books in the inventory.
2. View a specific book by its ID.
3. Search for books by title or author.
4. Add a new book to the inventory.
5. Update the price and stock of an existing book.
6. Delete a book from the inventory.

The API should be built using FastAPI and should include validation for incoming data.

Step 1: The in-memory database

We store our books in a Python list. This is a simple way to get started without setting up a full database.

Create main.py and add the following:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

app = FastAPI(
    title="Bookstore API",
    description="A simple API to manage a bookstore inventory",
    version="1.0.0"
)

# In-memory "database"
books_db = [
    {"id": 1, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "price": 12.99, "stock": 10},
    {"id": 2, "title": "To Kill a Mockingbird", "author": "Harper Lee", "price": 14.99, "stock": 5},
    {"id": 3, "title": "1984", "author": "George Orwell", "price": 9.99, "stock": 15},
    {"id": 4, "title": "Pride and Prejudice", "author": "Jane Austen", "price": 11.99, "stock": 7},
    {"id": 5, "title": "The Catcher in the Rye", "author": "J.D. Salinger", "price": 10.99, "stock": 8},
]
```

Step 3: Pydantic models

Define the data models for creating and updating books.

Add this code after the books_db list:

```
# Pydantic model for creating a new book
class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    author: str = Field(min_length=1, max_length=100)
    price: float = Field(gt=0, le=10000)
    stock: int = Field(ge=0, le=1000)

# Pydantic model for updating an existing book
class BookUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1, max_length=200)
    author: Optional[str] = Field(None, min_length=1, max_length=100)
    price: Optional[float] = Field(None, gt=0, le=10000)
    stock: Optional[int] = Field(None, ge=0, le=1000)
```

Step 3: The API endpoints

Now, let's build the endpoints one by one.

4.1 GET /books — Retrieve All Books

```
@app.get("/books")
def get_books():
    """
    Retrieve a list of all books in the inventory.
    """
    return books_db
```

Testing: Visit <http://127.0.0.1:8000/docs> and click GET /books. You will see the list of books in the response.

4.2 GET /books/{book_id} — Retrieve a Book by ID

```
@app.get("/books/{book_id}")
def get_book(book_id: int):
    """
    Retrieve a single book by its ID.
    """
    for book in books_db:
        if book["id"] == book_id:
            return book
    raise HTTPException(status_code=404, detail="Book not found")
```

Testing: Try /books/1 (should return the first book). Try /books/99 (should return a 404 error).

4.3 GET /books/search — Search for Books

```

@app.get("/books/search")
def search_books(q: str, author: Optional[str] = None):
    """
    Search for books by title or author.
    """
    results = []
    for book in books_db:
        # Check if the query string matches the title or author
        if q.lower() in book["title"].lower() or q.lower() in book["author"].lower():
            # Further filter by author if provided
            if author:
                if author.lower() in book["author"].lower():
                    results.append(book)
            else:
                results.append(book)
    return results

```

Testing: Try /books/search?q=gatsby (should return the first book). Try /books/search?q=orwell&author=george (should return "1984").

4.4 POST /books — Add a New Book

```

@app.post("/books")
def add_book(book: BookCreate):
    """
    Add a new book to the inventory.
    """
    # Check if a book with the same title and author already exists
    for existing_book in books_db:
        if existing_book["title"].lower() == book.title.lower() and existing_book["author"].lower()
        == book.author.lower():
            raise HTTPException(status_code=400, detail="Book already exists")

    # Generate a new ID
    new_id = max([b["id"] for b in books_db]) + 1 if books_db else 1

    # Create the new book
    new_book = {
        "id": new_id,
        "title": book.title,
        "author": book.author,
        "price": book.price,
        "stock": book.stock
    }

```

```

    }

    # Add to the database
    books_db.append(new_book)

    return {"message": "Book added successfully", "book": new_book}

```

Testing: Use Swagger UI to send a POST request with a JSON body:

```

{
  "title": "Dune",
  "author": "Frank Herbert",
  "price": 15.99,
  "stock": 12
}

```

4.5 PUT /books/{book_id} - update a book

```

@app.put("/books/{book_id}")
def update_book(book_id: int, book_update: BookUpdate):
    """
    Update an existing book's details.
    """

    # Find the book
    for index, book in enumerate(books_db):
        if book["id"] == book_id:
            # Update fields that are provided
            if book_update.title is not None:
                books_db[index]["title"] = book_update.title
            if book_update.author is not None:
                books_db[index]["author"] = book_update.author
            if book_update.price is not None:
                books_db[index]["price"] = book_update.price
            if book_update.stock is not None:
                books_db[index]["stock"] = book_update.stock

    return {"message": "Book updated successfully", "book": books_db[index]}

    raise HTTPException(status_code=404, detail="Book not found")

```

Testing: Send a PUT request to /books/1 with a JSON body like:

```

{
  "price": 13.99,
  "stock": 8
}

```

4.6 DELETE /books/{book_id} - delete a book

```
@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    """
    Delete a book from the inventory.
    """
    for index, book in enumerate(books_db):
        if book["id"] == book_id:
            # Remove the book from the list
            deleted_book = books_db.pop(index)
            return {"message": "Book deleted successfully", "book": deleted_book}

    raise HTTPException(status_code=404, detail="Book not found")
```

Testing: Send a DELETE request to /books/5 (should delete the book).

Exercises on the illustration

Exercise 1: Add a “Release Date” field

Task: Add a new optional field *release_date* to the Book data model (use datetime type). Update the BookCreate and BookUpdate models to include this field.

Exercise 2: Add pagination

Task: Add query parameters skip and limit to the GET /books endpoint to enable pagination:

- skip: number of books to skip (default 0)
- limit: maximum number of books to return (default 10)

Exercise 3: Add a “Category” field

Task: Add a new field category to the Book model (e.g., “Fiction”, “Non-Fiction”, “Science”). Update the relevant models and endpoints.

Project - Nairobi Freelance Gigs API

- The above work provided you with a template. Your project will have a different dataset and unique parameters.
- you cannot copy-paste this code. Your project has different field names and constraints.
- The techniques in the illustration above are what you will apply to your own problem solution below.

You are a backend developer hired by a Nairobi-based freelancing platform called *GigHub*. The platform connects freelancers (developers, designers, writers, etc.) with clients who need short-term work. The company needs an API to manage its gig listings. (*a gig = the specific record contains information about a freelance job*)

Project requirements

The API must allow clients and freelancers to:

1. List all available gigs (with optional filtering by category and min/max budget).
2. View details of a specific gig by its ID.
3. Create a new gig (clients post jobs).
4. Update a gig's budget or status (e.g., from “Open” to “In Progress”).
5. Delete a gig (if it is no longer available).

Part 1: Unique dataset (based on your admission number)

You will create a list of gigs. Your dataset is determined by your admission number.

Part of admission number	How it determines your data
Last digit (the last x in C027-01-xxxx/202y)	Determines the number of gigs you must create (5 + last digit).
The xxxx number (before the /2024)	Determines the categories you must use (if the xxxx is even, use [“Development”, “Design”, “Writing”]; if odd, use [“Marketing”, “Data”, “Consulting”]).
The first two digits of the xxxx number	Determines the currency for your budget field (KES if the first two digits are less than 10, USD if 10 or more).

Example:

- Admission Number: C027-01-1234/2024
 - Last digit: 4 → Create 5 + 4 = 9 gigs.
 - xxxx is 1234 (even) → Categories: [“Development”, “Design”, “Writing”].
 - First two digits: 12 → Currency: USD.

The data model (gig)

Each gig must have the following fields:

Field	Type	Constraints	Example
id	Integer	Auto-incremented	1
title	String	Min length 5, max length 100	“Build a React dashboard”
description	String	Min length 20, max length 500	“Build a React dashboard for a fintech startup...”
category	String	Must be one of your assigned categories	“Development”
budget	Float	Must be > 0	15000.0
currency	String	Determined by your admission number	“KES”
status	String	Must be one of: “Open”, “In Progress”, “Closed”	“Open”
client_name	String	Min length 2, max length 50	“Jane Muthoni”

Part 2: Creating the initial dataset

Based on your admission number, create a list of 5 + (last digit) gigs.

For admission number: C027-01-1234/2024 (Example)

Last digit = 4 → 5 + 4 = 9 gigs.

Categories = ["Development", "Design", "Writing"]

Currency = "KES"

gigs_db = [

 {"id": 1, "title": "Build a React Dashboard", "description": "Build a React dashboard for a fintech startup. Must be responsive and mobile-friendly.", "category": "Development", "budget": 15000.0, "currency": "KES", "status": "Open", "client_name": "Jane Muthoni"},

 # ... 8 more gigs to reach 9

]

Part 3: The API endpoints

implement the following endpoints:

Endpoint	Method	Description
/gigs	GET	Return all gigs (with optional filtering by category and min_budget/max_budget).
/gigs/{gig_id}	GET	Return a single gig by its ID (return 404 if not found).
/gigs/search	GET	Search for gigs by title (query param q).
/gigs	POST	Create a new gig (with validation).
/gigs/{gig_id}	PUT	Update a gig's budget or status.
/gigs/{gig_id}	DELETE	Delete a gig (return 404 if not found).

Part 4: Pydantic models

define two Pydantic models:

Model	Purpose	Fields
GigCreate	For creating a new gig	title, description, category, budget, client_name
GigUpdate	For updating an existing gig	budget (optional), status (optional)

Constraints:

- All fields in GigCreate must be validated (min/max lengths, positive budget, valid category).
- The status in GigUpdate must be one of "Open", "In Progress", or "Closed".

Part 5: Submission instructions

What to submit (20 marks)

1. GitHub repository link (5 marks):

- repository be named: gighub-api
- contains main.py with the complete API.
- includes a README.md file with a brief description of the API.

2. Swagger UI screenshot (5 marks):

- A screenshot of Swagger UI (/docs) showing all your endpoints.
- The screenshot must show your admission number somewhere (e.g., in the browser title bar or as a comment in the code).

3. Written responses (10 marks):

- Answer the questions in Part 6.

Anti-copying measures

Note: I will randomly select 5 students to do a 3-minute code walkthrough (viva). Besides, I will run MOSS (Measure of Software Similarity) to detect copied code.

Part 6: Written questions

Instructions: Answer these questions in your own words. Do not copy from a colleague. Each answer should be 2-3 sentences.

Question 1: design decisions (2 marks)

- (i) What is your admission number, and how did it determine your dataset (number of gigs, categories, and currency)?
- (ii) Why did you choose to include the status field in your gig data model? What are the possible statuses and what do they represent?

Question 2: Validation (2 marks)

- (i) What validation rules did you apply to the budget field in your Pydantic model? Why is this validation important?
- (ii) How did you ensure that the category field only accepts values from your assigned category list?

Question 3: Error handling (2 marks)

- (i) What status code does your API return when a user tries to retrieve a gig that does not exist? Explain why this is the appropriate status code.
- (ii) What status code does your API return when a user tries to create a gig with invalid data? Explain why.

Question 4: search and filtering (2 marks)

- (i) Describe how your `/gigs/search` endpoint works. How does it handle searches where the query string does not match any gig?
- (ii) Describe how your `/gigs` endpoint with filtering works. What are the optional query parameters, and how do they work?

Question 5: Code walkthrough (2 marks)

- (i) Write a brief explanation of the `PUT /gigs/{gig_id}` endpoint. What does it do, and how does it handle a request to update a gig that does not exist?
- (ii) Write a brief explanation of the `POST /gigs` endpoint. What does it do, and how does it ensure that the new gig is created correctly?